

Khaatiaraarot API

REST API for grocery e-commerce. Base path: /api/v1.

Quick Start

```
Base URL:    http://localhost:4000/api/v1
Swagger UI:  http://localhost:4000/api/v1/docs
```

All responses are JSON. Send Content-Type: application/json on POST/PUT/PATCH.

Always send requests with withCredentials: true (axios) / credentials: 'include' (fetch) so cookies flow correctly. This is required for refresh tokens, session-based guest carts, and cart merge on login.

Response Format

Success

```
{ "success": true, "data": { ... } }
```

Error

```
{ "success": false, "error": { "code": "ERR_CODE", "message": "Reason" } }
```

Common HTTP codes: 200 OK · 201 Created · 400 validation · 401 auth · 403 forbidden · 404 not found · 409 conflict · 429 rate limit · 500 server.

Read error.code for branching logic in your frontend. Read error.message for UI display.

Cookies the Server Sets

Cookie	Purpose	Path	Expiry
kha_refresh	Refresh token (secure)	(httpOnly, /api/v1/auth	30 days
kha_session	Guest cart identity (httpOnly)	/	30 days

Both are httpOnly — you cannot read them from JS. They flow automatically on every request when credentials: 'include' is set.

Authentication

JWT in Authorization header. Refresh token in httpOnly cookie.

```
Authorization: Bearer <accessToken>
```

Access token expires in 15 minutes. Use /auth/refresh to rotate.

Auth Flow

```
1. POST /auth/register OR POST /auth/login
   → Response: { accessToken, user }
   → Server sets: kha_refresh cookie

2. Store accessToken in memory (not localStorage).

3. Attach on every protected request:
   Authorization: Bearer <accessToken>

4. On 401 response:
   → POST /auth/refresh (cookie sends kha_refresh automatically)
   → Response: new accessToken + new kha_refresh cookie
   → Retry original request with new token.

5. POST /auth/logout
   → Server clears kha_refresh cookie.
   → Discard accessToken from memory.
```

Roles

- customer — default for all registered users.
- admin — required for all /admin/* routes. Non-admin gets 403.

Guest Cart & Cart Merge on Login

The cart system works for both guests and logged-in users with zero configuration on the frontend. Here is exactly what happens:

How it works

Guest (not logged in)

- On the first cart request (GET /cart, POST /cart/items, etc.), the server creates a kha_session cookie with a random UUID if one doesn't already exist.
- All cart operations are tied to this session ID server-side.
- The session cookie lives for 30 days.
- No Authorization header needed for any cart endpoint.

Logged-in user

- Cart operations use userId from the Bearer token instead of the session cookie.
- The user has their own persistent cart in the database.

Merge on Login

When a guest adds items to the cart and then logs in, the guest cart is automatically merged into the user cart during the login response:

```
1. Guest adds products → cart stored under kha_session cookie UUID.

2. Guest hits POST /auth/login (credentials: 'include' must be set).

3. Server reads kha_session cookie, merges guest cart into user cart:
   - If product already in user cart → quantities are ADDED together,
     capped at available stock.
   - If product only in guest cart → moved to user cart.
   - Guest cart is then deleted.
   - kha_session cookie is cleared.

4. From this point, all cart calls use the Bearer token / userId.
```

Frontend implementation checklist:

- Always send credentials: 'include' on all requests (not just auth).
- Do NOT manually handle session IDs — the cookie does it automatically.
- After login, store the accessToken from the response and attach it as Authorization: Bearer.
- After login, GET /cart will return the merged cart.
- On logout, the user cart persists in the database. Next login restores it.

Example: guest adds item, then logs in

```
// Guest — no token needed, cookie handles identity
await api.post('/cart/items', { productId: 'uuid', quantity: 2 });

// Login — merges guest cart automatically
const { data } = await api.post('/auth/login', { email, password });
const token = data.data.accessToken;
api.defaults.headers.common.Authorization = `Bearer ${token}`;

// Fetch merged cart
```

```
const cart = await api.get('/cart');
// cart now contains items from both guest session + any prior user cart
```

Complete Checkout Workflow

Full end-to-end flow from product discovery to order placed:

1. Browse
 - GET /products → list products (paginated, filterable)
 - GET /products/:id → product detail with images
 - GET /categories → all active categories
 - GET /banners?type=hero → homepage banners
2. Cart (works as guest or logged-in)
 - POST /cart/items → add to cart { productId, quantity }
 - GET /cart → view cart with product details + subtotal
 - PATCH /cart/items/:productId → update quantity
 - DELETE /cart/items/:productId → remove item
3. Login / Register (triggers cart merge if guest had items)
 - POST /auth/login → { accessToken, user }
 - [Attach Bearer token to all subsequent requests]
4. (Optional) Apply Coupon
 - POST /coupons/validate → { code, orderAmount }
 - returns { coupon, discountAmount }
 - [Store coupon code, apply discount in UI, pass couponCode in order body]
5. Checkout – collect delivery address
 - GET /districts → dropdown list of Bangladesh districts
6. Place Order
 - POST /orders
 - Body: {
 - paymentMethod: "bkash" | "nagad" | "cash" | "card" | "gcash" | "manual",
 - address: {
 - fullName, phone, line1, line2?,
 - city, district, postalCode?
 - },
 - notes?,
 - couponCode? ← if coupon was validated in step 4
 - Header: Idempotency-Key: <uuid> ← generate once per checkout attempt
 - 201: { order }

→ Cart is cleared automatically after order is placed.

7. Confirmation

GET /orders/:id → full order with items, totals, status

8. Track Orders

GET /orders?page=1&limit=10 → list all user orders

PATCH /orders/:id/cancel → cancel if status is pending/confirmed

Delivery Cost

Products have an optional ratePlanId. When district is selected in checkout:

- The order service looks up the rate plan for that product's district.
- Delivery cost is calculated per product based on $\text{costPerUnit} \times \text{quantity}$.
- If no rate plan is assigned to a product, delivery for that item is free.
- Frontend does not need to calculate delivery — the server handles it and returns the final total in the order response.

Payment Methods

cash · card · gcash · bkaash · nagad · manual

For Bkash/Nagad/card — the current system records the payment method but does not process payment inline. Payment verification is done manually by admin (PUT /admin/orders/:id/status).

Endpoints

Auth /auth

Method	Path	Auth	Purpose
POST	/register	—	Create customer. Body: fullName, email, password, phone
POST	/login	—	Body: email, password. Returns accessToken + merges guest cart
POST	/refresh	cookie	Rotate refresh token, get new access token
POST	/logout	bearer	Revoke tokens, clear refresh cookie

GET /me bearer Get authenticated user profile

Register / Login response shape:

```
{
  "success": true,
  "data": {
    "accessToken": "eyJ...",
    "user": {
      "id": "uuid",
      "email": "user@example.com",
      "fullName": "Name",
      "role": "customer"
    }
  }
}
```

Products /products (public)

Method	Path	Purpose
GET	/	List. Query: page, limit, q, category price_asc\ (slug), sort (`newest\
GET	/top-sellers	Best-selling products
GET	/:id	Single product with images and category (UUID)

Categories /categories (public)

Method	Path	Purpose
GET	/	All active categories with sortOrder
GET	/:slug	Category by slug

Banners /banners (public)

Method	Path	Purpose
GET	/	Active banners. Query: type (`hero\ side\

Districts /districts (public)

Method	Path	Purpose
GET	/	Bangladesh district list — use for address form dropdown

Cart /cart (guest or authenticated — no bearer required)

Cart endpoints accept both guests (via kha_session cookie) and authenticated users (via Authorization header). Always send credentials: 'include'.

Method	Path	Purpose
GET	/	Current cart with items, product details, subtotal, itemCount
POST	/items	Add item. Body: productId, quantity
PATCH	/items/:productId	Update qty. Body: quantity (set to 0 to remove)
DELETE	/items/:productId	Remove item
DELETE	/	Clear entire cart

Cart response shape:

```
{
  "success": true,
  "data": {
    "id": "uuid",
    "items": [
      {
        "id": "cart-item-uuid",
        "quantity": 2,
        "product": {
          "id": "uuid",
          "name": "Kataribhog Rice",
          "slug": "kataribhog-rice",
          "unit": "kg",
          "price": 120.00,
          "originalPrice": 140.00,
          "stockQty": 500,
          "image": "https://res.cloudinary.com/..."
        }
      }
    ],
    "itemCount": 2,
    "subtotal": 240.00
  }
}
```

```
}  
}
```

Wishlist /wishlist (auth required)

Method	Path	Purpose
GET	/	Wishlist with full product details
POST	/items	Add product. Body: productId. Idempotent — safe to call even if already added
DELETE	/items/:productId	Remove product

Rewards /rewards (auth required)

Method	Path	Purpose
GET	/	Points balance and recent transactions

Response shape:

```
{  
  "success": true,  
  "data": {  
    "balance": 150,  
    "lifetimeEarned": 320,  
    "transactions": [  
      {  
        "id": "uuid",  
        "type": "earn",  
        "points": 50,  
        "description": "Order #KHA-00042",  
        "createdAt": "2026-06-15T10:00:00Z"  
      }  
    ]  
  }  
}
```

Coupons /coupons (auth required)

Method	Path	Purpose
POST	/validate	Validate code before checkout. Body: code, orderAmount

Validate response:

```
{
  "success": true,
  "data": {
    "coupon": {
      "id": "uuid",
      "code": "SAVE10",
      "type": "percentage",
      "value": 10
    },
    "discountAmount": 75.00
  }
}
```

Errors on validate: expired, usage limit reached, per-user limit reached, order amount below minimum.

Orders /orders (auth required)

Method	Path	Purpose
POST	/	Place order from cart
GET	/	List own orders. Query: page, limit
GET	/:id	Order detail with items and status history
PATCH	/:id/cancel	Cancel order — only allowed when status is pending or confirmed

Order statuses: pending → confirmed → processing → shipped → delivered

Cancel allowed from: pending, confirmed

Terminal statuses: delivered, cancelled, refunded

Idempotency: Pass a unique Idempotency-Key header on POST /orders. If the same key is sent twice (e.g. network retry), the server returns the original order instead of creating a duplicate. Generate a new UUID per checkout attempt, not per session.

Admin /admin (admin role required)

All admin routes require Authorization: Bearer <adminToken>. Non-admin → 403 Forbidden.

Products /admin/products

Method	Path	Purpose
GET	/	All products. Query: page, limit, q, categoryId, isActive
POST	/	Create. Body: name, slug, unit, price, description?, sourceRegion?, categoryId?, ratePlanId?, originalPrice?, stockQty?, lowStockThreshold?, isBestSelling?, isActive?
PUT	/:id	Partial update (any field from create)
DELETE	/:id	Soft delete (isActive = false)
PUT	/:id/stock	Set stock directly. Body: stockQty
POST	/:id/images	Upload image. multipart/form-data, field name image
DELETE	/:id/images/:imageld	Remove image from Cloudinary
PATCH	/:id/images/:imageld/primary	Set as primary image

Categories /admin/categories

Method	Path	Purpose
GET	/	All (including inactive)
POST	/	Body: name, slug, nameBn?, sortOrder?, isActive?
PUT	/:id	Update
DELETE	/:id	Soft delete
POST	/:id/image	Upload category image (multipart)

Banners /admin/banners

Method	Path	Purpose
GET	/	All banners (including inactive/scheduled)
POST	/	Body: `type(hero\` side\`
PUT	/:id	Update
DELETE	/:id	Hard delete
POST	/:id/image	Upload banner image (multipart)

Orders /admin/orders

Method	Path	Purpose
GET	/	All orders. Query: page, limit, status, q (search), from, to (ISO date)
POST	/	Manual order. Body: `source(facebook\ phone\`
GET	/:id	Full detail with history and all line items
PUT	/:id/status	Update status. Body: status, note?
POST	/:id/invoice	Queue invoice PDF generation and email to customer

Inventory /admin/inventory

Method	Path	Purpose
GET	/	Stock levels. Query: page, limit, lowStockOnly (boolean)

Rate Plans /admin/rate-plans

Delivery cost per district. Products are assigned a rate plan — the plan's per-district cost drives shipping calculations at checkout.

Method	Path	Purpose
GET	/	All plans with district rates
POST	/	Body: name, description?, isActive?, rates[{district, costPerUnit}]
GET	/:id	Single plan with all district rates
PUT	/:id	Update. Providing rates array replaces all existing district rates
DELETE	/:id	Delete plan — sets ratePlanId = null on all linked products

Coupons /admin/coupons

Method	Path	Purpose
GET	/	All coupons. Query: page, limit, isActive
POST	/	Create. Body: `code, type(percentage\ fixed), val maxDiscount? perUserLimit?`
GET	/:id	Single coupon with usage stats
PUT	/:id	Update (any field from create)
DELETE	/:id	Hard delete

Coupon fields:

- type: percentage — value is percent (e.g. 10 = 10% off). maxDiscount caps the deduction.
- type: fixed — value is flat BDT amount off.
- minOrderAmount — coupon invalid if cart subtotal is below this.
- usageLimit — total redemptions allowed across all users.
- perUserLimit — defaults to 1 (one use per user).

Reports /admin/reports

Method	Path	Purpose
GET	/dashboard	Revenue + order counts for today/month/year/all-time, low stock count, recent orders
GET	/sales	Time series revenue. Query: from, to month` (ISO), `group(day\`
GET	/revenue	Revenue by payment method and order source. Query: from, to
GET	/top-products	Top products by revenue. Query: from, to, limit (1–50, default 10)
GET	/top-categories	Top categories by revenue. Query: from, to

Integration Notes

Setup (required)

```
import axios from 'axios';

export const api = axios.create({
  baseURL: 'http://localhost:4000/api/v1',
  withCredentials: true, // REQUIRED – do not omit
});
```

Token Interceptor

```
// Attach token
api.interceptors.request.use(config => {
  const token = getAccessToken(); // from your auth store
```

```

    if (token) config.headers.Authorization = `Bearer ${token}`;
    return config;
  });

  // Auto-refresh on 401
  api.interceptors.response.use(
    r => r,
    async err => {
      if (err.response?.status === 401 && !err.config._retry) {
        err.config._retry = true;
        try {
          const r = await api.post('/auth/refresh');
          const newToken = r.data.data.accessToken;
          setAccessToken(newToken); // update your auth store
          err.config.headers.Authorization = `Bearer ${newToken}`;
          return api(err.config);
        } catch {
          clearAuth(); // refresh failed – redirect to login
          throw err;
        }
      }
      throw err;
    }
  );

```

CORS

Server allows the origin set in ALLOWED_ORIGIN env var. Must match exactly — no trailing slash.

Rate Limiting

- /auth/* routes have a stricter rate limiter.
- Image upload routes have a separate upload rate limiter.
- On 429: back off and retry after the Retry-After header value.

Images

Stored in Cloudinary. Responses return full CDN URLs — use directly in . No signed URL or proxy needed.

Pagination

```

{
  "success": true,

```

```
"data": [...],
"pagination": {
  "page": 1,
  "limit": 20,
  "total": 83,
  "totalPages": 5
}
```

IDs

All IDs are UUIDs unless stated otherwise.

Dates

All timestamps are ISO 8601 strings with timezone (2026-06-15T10:00:00.000Z). Send startsAt/endsAt/expiresAt/from/to in the same format.

Full Reference

Swagger UI lists every field, enum, and response shape: <http://localhost:4000/api/v1/docs>